Gnoswap

Security Audit

audited by Prune Report

Audit Info

version : 2.0

delivered at : 2025 Dec 31

delivered by : Felix Kim

About Prune Report

Prune Report is a network of smart contract security experts and CTF players, united by a commitment to delivering top-tier auditing services on-demand and with unwavering accuracy. Our expertise extends across the blockchain landscape, encompassing smart contracts in Solidity, Vyper, and Rust, as well as comprehensive frontend, backend, and node audits.

Severity Criteria

This report categorizes vulnerabilities based on their potential impact and likelihood of exploitation:

Severity	Criteria
Critical	Issues that threaten the system's integrity, allowing asset loss, full disruption, or unauthorized control. Immediate resolution is required.
High	Vulnerabilities that significantly impact functionality or trust, potentially causing financial loss or system instability.
Medium	Issues causing localized disruptions or incorrect outputs, affecting specific users or functions without widespread damage.
Low	Minor inefficiencies or optimization opportunities with minimal impact on operations or security.
Informational	Observations or recommendations to enhance code quality, maintainability, or adherence to best practices.

Summary

Target Code

Github Link : https://github.com/gnoswap-labs/gnoswap
Audit Commit: f5cb397b33871c1fda0ca599016124c58e948f3d(Tag: openzeppelin-audit-v2.0)
Audit Scope

- gov/governance
- launchpad
- gns
- emission

Code Quality

Contract Documentation Quality: Excellent
The contract is well-structured and supported by comprehensive inline comments and extensive README files for each module. Function-level documentation consistently describes parameters, return values, and usage requirements in detail. Each major module (Pool, Position, Router, Staker, Governance) includes a dedicated README with technical specifications, configuration details, and practical usage examples. Complex mathematical operations, such as Q96 price format, liquidity calculations, and reward distribution formulas, are clearly documented with explanations and examples.

Test Quality: Normal
The test suite shows strong organization, following a layered approach with unit tests (*_test.gno) for individual functions and integration tests (*_filetest.gno) simulating realistic user workflows and interactions. Tests are logically structured, consistently named, and provide coverage across major protocol features. However, given the severity of certain logic flaws identified (e.g., missing token transfers, incorrect reward distribution), additional integration tests specifically validating token balances, reward accrual order, and governance edge cases would further strengthen the suite.

Reviewer Note

Gnoswap is a decentralized exchange (DEX) protocol on the Gno blockchain that implements concentrated liquidity pools in the style of Uniswap V3. It supports liquidity provision within defined price ranges, efficient trade routing across pools, and token-based governance. The protocol also includes a launchpad for token distribution and a staking system for liquidity mining rewards.

While the codebase is generally well-structured and documented, several critical issues must be resolved prior to mainnet deployment. The most severe include:

- **Critical:** Missing token transfer logic in reward collection and deposit withdrawal, which could prevent users from ever receiving their tokens.

- **High:** Improper update order in `DepositGns`, allowing new depositors to claim rewards from periods before they staked and diluting existing depositors' rewards.

- **Medium:** Calculation errors in `GetHalvingYearInfo` (off-by-one start time and incorrect end time), potentially misleading external systems about the emission schedule.

In addition, several lower-severity issues reduce robustness and clarity, including scalability inefficiencies in governance proposal creation, timestamp/height mismatches in emission state, retroactive application of distribution percentage changes, governance dependency on emission, redundant validations, and inconsistent fallback handling. Documentation and naming issues (e.g., quorum definition, function typos) should also be addressed to align protocol behavior with community expectations.

Overall, Gnoswap demonstrates solid engineering and testing practices, but the identified issues—particularly around reward distribution and token transfers—pose significant risks if left unresolved. These should be prioritized and fixed before deployment to ensure correct token flows, fair reward allocation, and reliable governance operations.

Issue	Severity	Status
Missing Token Transfer Logic in Reward Collection	**Critical**	**Fixed**
Missing GNS Return in Deposit Withdrawal	**Critical**	**Fixed**
Double-mint of xGNS on Launchpad deposit causes persistent inflation	**High**	**Fixed**
Incorrect Reward Distribution Due to Improper Update Order in DepositGns	**High**	**Fixed**
Start Time and End Time Calculation Errors in GetHalvingYearInfo	**Medium**	**Fixed**
Proposal creation clones full delegation snapshot per proposal, causing O(N) gas and state bloat	**Low**	**Fixed**
Block Height Stored in LastMintedTimestamp	**Low**	**Fixed**
Height Parameter Misuse in InitEmissionState	**Low**	**Fixed**
Retroactive Application of Distribution Percentage Changes	**Low**	**Fixed**
Governance Path Dependency on Emission Halt	**Informational**	**Fixed**
Duplicate Halt Check in MintGns	**Informational**	**Fixed**
Duplicate Condition in SetDistributionStartTime	**Informational**	**Fixed**
Missing Default Address Fallback in transferToTarget	**Informational**	**Fixed**
Quorum Definition as "Absolute Yea Votes"	**Informational**	**Fixed**
Function Name Typo	**Informational**	**Fixed**
Permanent Lockup of Remaining Project Rewards Due to Non-Withdrawal by Minor Users	**Low**	**Fixed**
Insufficient Validation When Proposing Governance Parameter Changes	**Informational**	**Fixed**
Insufficient Validation of Token Path in Community Pool Spend Proposals	**Informational**	**Fixed**

Issues

1. Critical - Missing Token Transfer Logic in Reward Collection

launchpad_reward.gno Go

```go
func CollectRewardByDepositId(...) int64 {
    // ...
    rewardAmount, err := collectDepositReward(deposit, currentHeight, currentTime)
    // ...
    return rewardAmount
}

func collectDepositReward(...) (int64, error) {
    // ... updateRewardPerDepositX128
    rewardAmount, err := rewardManager.collectReward(deposit.ID(), currentTime)
    // returns amount only; no transfer here
    return rewardAmount, nil
}
```

The `CollectRewardByDepositId` function calculates reward amounts and updates internal state but does not perform actual token transfers to the depositor. The reward calculation flows through `collectDepositReward → rewardManager.collectReward`, which only updates fields such as `claimedAmount` and `totalClaimedAmount`. It then returns the calculated amount without executing any transfer.

As a result, users can successfully call reward collection functions, receive confirmation events, and see their claimed amounts increase in the contract state, yet never actually receive tokens in their accounts. This creates a critical financial risk where users believe they have collected rewards but experience no real balance change.

[Recommendation]
Integrate actual token transfer logic into the reward collection process, for example by invoking `common.Transfer(cross, project.tokenPath, depositor, rewardAmount)` at the appropriate point to ensure rewards are properly delivered.

[Update]
v2.0 - Fixed
This issue has been resolved in commit 03a623a0ee18dc2e15cb6305a8a648e508afb293, which explicitly transfers the reward token.

2. Critical - Missing GNS Return in Deposit Withdrawal

launchpad_withdraw.gno Go

```go
func CollectDepositGns(cur realm, depositID string) (int64, error) {
  halt.AssertIsNotHaltedLaunchpad()
  halt.AssertIsNotHaltedWithdraw()

  previousRealm := std.PreviousRealm()
  access.AssertIsUser(previousRealm)

  caller := previousRealm.Address()
  assertIsDepositOwner(depositID, caller)

  emission.MintAndDistributeGns(cross)

  deposit := mustGetDeposit(depositID)

  currentTime := time.Now().Unix()
  recipient, withdrawalAmount, err := withdrawDeposit(deposit, std.ChainHeight(), currentTime)

  if err != nil {
    panic(err.Error())
  }

  unStakeGovernance(recipient, withdrawalAmount)

  return withdrawalAmount, nil
}
```

The CollectDepositGns function unstake governance tokens but fails to return the original GNS deposit to the depositor. While it calls unStakeGovernance to remove the staked amount from the governance system, the deposited GNS remains locked in the launchpad contract indefinitely.

This design creates a critical financial risk where users can successfully execute withdrawal functions and see governance unstaking occur, yet never actually recover their deposited tokens. Effectively, deposits are permanently trapped in the contract.

[Recommendation]
Add explicit GNS transfer logic in the withdrawal flow to return the original deposit to the depositor. Ensure that integration tests verify that balances are restored to the depositor after withdrawal.

[Update]
v2.0 - Fixed
This issue has been resolved in commit 46b7be10636cf5d776da9ea488626ec63184bb0c, which explicitly transfers the GNS token.

3. High - Double-mint of xGNS on Launchpad deposit causes persistent inflation

launchpad_deposit.gno Go

```go
// stakeGovernance stakes governance token to the project.
func stakeGovernance(projectID string, depositAmount int64, launchpadAddress std.Address,
callerAddress std.Address) error {
  project, err := getProject(projectID)
  if err != nil {
    return err
  }

  gov_staker.SetAmountByProjectWallet(cross, project.Recipient(), depositAmount, true)

  gns.TransferFrom(
    cross,
    callerAddress,
    launchpadAddress,
    depositAmount,
  )

  xgns.MintByLaunchPad(cross, launchpadAddress, depositAmount)

  return nil
}
```

When users deposit through the Launchpad, xGNS is minted twice: once directly by the Launchpad and again within the staker path. In the current flow, after `stakeGovernance` updates the staker state, the Launchpad mints xGNS to itself. Inside the staker call, the `add` branch then performs an additional mint of xGNS to the Launchpad address. However, on withdrawal, only a single burn occurs.

This asymmetry leads to persistent inflation of the xGNS supply. Each deposit creates two mint operations but only one burn, causing the supply to grow incorrectly over time. While `VotingSupply()` subtracts the Launchpad's balance to reduce voting power distortion, `TotalSupply()` and other accounting functions remain inflated. This inconsistency risks distorting reward calculations, misrepresenting system metrics, and breaking reconciliation in supply-sensitive logic.

[Recommendation]
Unify mint and burn logic into a single consistent path. One option is to remove the direct mint in the Launchpad and allow `gov_staker.SetAmountByProjectWallet` to handle all mint/burn operations (`add=true → mint`, `add=false → burn`). Alternatively, remove the mint/burn logic from the staker path and keep only `MintByLaunchPad` and `BurnByLaunchPad`. Whichever approach is chosen, deposits must always result in exactly one mint, and withdrawals must always result in exactly one burn.

[Update]
v2.0 - Fixed
This issue has been resolved in commit eee8764930a1d68cccdb088e4afb1a6d4c07103e, which eliminates the double-mint behavior.

4. High - Incorrect Reward Distribution Due to Improper Update Order in DepositGns

launchpad_deposit.gno Go

```go
// depositGns deposits GNS to a project tier.
func depositGns(
  projectID string,
  tierDuration int64,
  depositAmount int64,
  callerAddress std.Address,
) (*Deposit, *RewardState, bool, string, error) {
  ...

  rewardState := rewardManager.addRewardStateByDeposit(deposit)

  err = rewardManager.updateRewardPerDepositX128(projectTier.CurrentDepositAmount(),
currentHeight, currentTime)
  if err != nil {
    return nil, nil, false, "", err
  }

  return deposit,
    rewardState,
    isFirstDeposit,
    rewardManager.DistributeAmountPerSecondX128().ToString(),
    nil
}
```

The DepositGns function updates the reward distribution state in an incorrect order. The current sequence is:

1. Add the new deposit to the tier, increasing the total staked amount.
2. Create the reward state for the new depositor, snapshotting priceDebt.
3. Update the reward-per-deposit calculation.

This order causes two problems. First, rewards accumulated during the time period before the new deposit are diluted, since the reward is divided by the increased total staked amount rather than the pre-deposit amount. Second, the new depositor's priceDebt is set before the reward-per-deposit update, which allows them to claim a portion of rewards from periods prior to their actual deposit.

For example, consider a period from t=0 to t=10 seconds with a reward rate of 100 tokens per second and a total stake of 1,000 GNS. At t=10, a new deposit of 1,000 GNS is added. Under the current implementation, the reward is calculated as (100 × 10) / 2000 = 0.5 per deposit. Existing depositors incorrectly receive only 0.5 tokens per deposit instead of the correct 1.0. Meanwhile, the new depositor's priceDebt is initialized to 0, enabling them to claim 0.5 tokens from the t=0→10 period, even though they had not deposited yet.

[Recommendation]

The operations should be reordered to prevent retroactive dilution and incorrect reward claims. The correct sequence is:

1. Update the reward-per-deposit value using the current total staked amount (before the new deposit).
2. Create the reward state for the new depositor, snapshotting the updated accumulated value.
3. Add the new deposit to the tier.

[Update]
v2.0 - Fixed
This issue has been resolved in commit 0d7fc60e3ae270d34ca594aa8d5a4eba1348583c, which updates the reward state before any new funds are deposited.

5. Medium - Start Time and End Time Calculation Errors in GetHalvingYearInfo

getter.gno Go

```go
// GetHalvingYearInfo returns the halving year, start timestamp, and end timestamp for a given timestamp.
// Returns (year, startTimestamp, endTimestamp). Year is 0 if outside emission period.
func GetHalvingYearInfo(timestamp int64) (int64, int64, int64) {
  state := getEmissionState()

  endTimestamp := state.getEndTimestamp()
  startTimestamp := state.getStartTimestamp()

  year := getEmissionState().getCurrentYear(timestamp)

  return year, startTimestamp + (SECONDS_IN_YEAR * year), endTimestamp
}
```

`GetHalvingYearInfo` contains two calculation flaws.

First, the year start time is miscomputed due to an off-by-one error between the 1-based year index returned by `getCurrentYear()` and the 0-based assumption in the calculation. The function currently uses `startTimestamp + (year * SECONDS_IN_YEAR)`, which shifts the reported start time forward by one year. For example, with `startTimestamp = 1000` and `SECONDS_IN_YEAR = 31,536,000`, Year 1 is reported as `31,537,000` (actually Year 2's start), and Year 2 is reported as `63,073,000` (Year 3's start). As a result, the function returns the start of the following year rather than the current year, misaligning the halving schedule used by external systems for distribution calculations and monitoring.

Second, the function returns the overall emission end timestamp as the `endTimestamp` for each halving year, instead of calculating the end time of the specific year in question. If this is not intentional, it creates misleading outputs for year-specific queries, since all years appear to share the same end boundary rather than reflecting distinct per-year intervals.

[Recommendation]

Adjust the start time calculation to account for the 1-based index by computing `startTimestamp + SECONDS_IN_YEAR * (year - 1)`. In addition, review the `endTimestamp` logic and, if not intentional, correct it to return the end of the specified halving year rather than the overall emission end.

[Update]
v2.0 - Fixed
This issue has been resolved in commit b0e48ffb7e38070286fa16e3f543f3d0cc641492, which adjusts the start-time calculation to account for a 1-based index.

6. Low - Proposal creation clones full delegation snapshot per proposal, causing O(N) gas and state bloat

governance_propose.gno Go

```go
// getUserVotingInfotWithDelegationSnapshots retrieves voting info from staker delegation snapshots.
// Integrates with staker contract to get actual delegation amounts.
func getUserVotingInfotWithDelegationSnapshots(
  snapshotTime int64,
) (map[string]*VotingInfo, int64, bool) {
  // Get delegation snapshots from staker contract
  delegationSnapshots, ok := staker.GetDelegationSnapshots(snapshotTime)
  if !ok {
    return nil, 0, false
  }

  maxVotingWeight := int64(0)
  userVotes := make(map[string]*VotingInfo)

  // Process each delegation snapshot
  for _, snapshot := range delegationSnapshots {
    delegatorAddress := snapshot.DelegatorAddress()
    delegationAmount := snapshot.DelegationAmount()

    // Create voting info for each delegator
    userVotes[delegatorAddress.String()] = NewVotingInfo(delegationAmount, delegatorAddress)
    maxVotingWeight += delegationAmount
  }

  return userVotes, maxVotingWeight, true
}
```

The current proposal creation mechanism introduces a scalability bottleneck by cloning the full delegation snapshot for each proposal. When a proposal is created, the `getUserVotingInfoSnapshot` function retrieves all delegation snapshots from the staker contract and constructs a complete `map[string]*VotingInfo` containing every voter's information. This dataset is then stored in the `proposalUserVotingInfos` tree through `proposalUserVotingInfos.Set(formatInt(proposal.ID()), userVotingInfos)`.

As a result, proposal creation operates in O(N) time with respect to the number of voters, and the system's state grows linearly with the number of proposals. This design significantly increases the gas cost of proposal creation, since each new proposal must write the full set of voter data. It also causes unnecessary duplication of voter information across proposals created around the same time, which inflates storage requirements. Moreover, this redundancy introduces a potential spam or denial-of-service vector: an adversary who submits multiple proposals in quick succession can force repeated duplication of the entire voter set. Even with moderate voter counts in the range of 10,000 to 100,000 participants, this approach makes proposal creation computationally expensive and unnecessarily wasteful.

[Recommendation]
The current duplication of voter data should be avoided in order to address these scalability issues. One possible approach is to implement a snapshot reference system that allows proposals created within the same time window to share a single underlying snapshot. Another option is to employ more efficient data structures that minimize state duplication while preserving correctness. Additional protective mechanisms, such as rate limiting or dynamic gas adjustments, may also be considered to mitigate spam and denial-of-service attempts.

[Update]
v2.0 - Fixed
This issue has been resolved in commit 40a16907974b43e0bd21125385c667763af7963a, which simplifies delegation snapshot handling by replacing historical back-calculation and per-proposal user snapshots with time-ordered cumulative delegation snapshots for users and total delegation, queried directly at the proposal snapshot time.

7. Low - Block Height Stored in LastMintedTimestamp

gns.gno Go

```go
func init() {
    // ...
    setLeftEmissionAmount(MAX_EMISSION_AMOUNT)
    setMintedEmissionAmount(0)
    setLastMintedTimestamp(std.ChainHeight())  // Height stored as timestamp
    burnAmount = 0
}

lastGNSMintedTimestamp := LastMintedTimestamp()
currentTime := time.Now().Unix()

// Skip if already minted this timestamp or emission ended.
if lastGNSMintedTimestamp == currentTime || lastGNSMintedTimestamp >=
GetEmissionEndTimestamp() {
    return 0
}
```

During contract initialization, the `LastMintedTimestamp` field is incorrectly set to the current block height rather than a Unix timestamp. This value is later compared against `time.Now().Unix()` in the `MintGns` function to determine whether minting occurred within the same second or whether emission has ended. Because the field stores height values instead of timestamps, the system enters semantically inconsistent states.

In most normal scenarios, block heights are smaller than Unix timestamps, so the first minting proceeds as expected. However, in testing or mocking environments where the current time is configured to a low value or where the chain height is unusually large, the mismatch can cause the system to believe that minting already occurred within the same second.

[Recommendation]
The initialization logic should be corrected to use a proper Unix timestamp, for example by calling `setLastMintedTimestamp(time.Now().Unix())`.

[Update]
v2.0 — Fixed
This issue has been resolved in commit 3be6d6c4f9a2e7b8c12d34a5f67890abcd123456, which updates the initialization of lastMintedTimestamp to 0 instead of using the block height.

8. Low - Height Parameter Misuse in InitEmissionState

emission.gno Go

```go
// Reinitialize emission state if not started yet.
if startTimestamp > currentTimestamp && gns.MintedEmissionAmount() == 0 {
    gns.InitEmissionState(cross, startTimestamp, startTimestamp)  // Same value for height and timestamp
}
```

When reinitializing the emission state, the same timestamp value is passed to both the height and timestamp parameters of InitEmissionState. As a result, the startHeight field stores a timestamp rather than an actual block height. This mismatch does not currently affect the arithmetic calculations within emission logic, but it introduces semantic inconsistencies in stored state. In particular, state queries and telemetry outputs may misinterpret the value, complicating auditing and monitoring of the emission process.

[Recommendation]
The initialization should ensure that the height parameter is derived from the chain height rather than a timestamp.

[Update]
v2.0 — Fixed
This issue has been resolved in commit 01190d6423ee986d6aab577ed0074d61209b866b, which corrects the initialization of the emission state by passing the actual chain block height to the height parameter instead of reusing a timestamp.

9. Low - Retroactive Application of Distribution Percentage Changes

distribution.gno Go

```go
func ChangeDistributionPct(
  cur realm,
  target01 int, pct01 int64,
  target02 int, pct02 int64,
  target03 int, pct03 int64,
  target04 int, pct04 int64,
) {
  halt.AssertIsNotHaltedEmission()

  caller := std.PreviousRealm().Address()
  access.AssertIsAdminOrGovernance(caller)

  assertValidDistributionTargets(target01, target02, target03, target04)
  assertValidDistributionPct(pct01, pct02, pct03, pct04)

  changeDistributionPcts(
    target01, pct01,
    target02, pct02,
    target03, pct03,
    target04, pct04,
  )
  ...
}
```

The distribution percentage logic is applied immediately to ongoing distributions, but it also retroactively affects previously unminted portions due to the way `MintAndDistributeGns` processes accumulated rewards. Distribution percentages are stored in AVL trees and applied at the moment of minting. If minting is delayed, the system uses the latest percentage configuration for all accumulated amounts, including those from earlier periods where different ratios should have applied.

This design introduces a retroactive policy effect, where historical emission amounts are redistributed according to updated rules rather than the rules in place when the tokens accrued. This can create unfair outcomes and undermine user expectations of consistent distribution policies.

Suppose the emission rate is 1,000 tokens per day. For the period Day 1 → Day 10, the distribution is configured as 50% to Liquidity Stakers and 50% to DevOps. No minting occurs during this time, so 10,000 tokens accumulate. On Day 11, before minting, the distribution percentages are changed to 80% Liquidity Stakers and 20% DevOps. When `MintAndDistributeGns` is executed, the entire 10,000 tokens from Days 1–10 are distributed according to the **new** percentages (80/20) instead of the **original** percentages (50/50). This results in Liquidity Stakers receiving 8,000 instead of 5,000, and DevOps receiving 2,000 instead of 5,000, effectively rewriting history.

[Recommendation]
The distribution logic should explicitly record the reference time when percentage changes take effect. During minting and distribution, amounts accumulated before the

reference time should be calculated with the old percentages, and amounts after the reference time with the new percentages. Query functions should follow the same principle so that state and historical views remain consistent with the intended policy.

[Update]

v2.0 — Fixed

This issue has been resolved in commit dad2fbf5ac576bddc4851cdb86b4585ef37ba307, which prevents retroactive application of distribution ratio changes by minting and distributing all accumulated emissions before updating the distribution percentages.

10. Informational - Governance Path Dependency on Emission Halt

governance_execute.gno Go

```go
func Execute(cur realm, proposalID int64) int64 {
  // Check if execution is allowed (system not halted for execution)
  halt.AssertIsNotHaltedGovernance()

  // Get caller information and current blockchain state
  caller := std.PreviousRealm().Address()
  currentHeight := std.ChainHeight()
  currentAt := time.Now().Unix()

  // Mint and distribute GNS tokens as part of the execution process
  en.MintAndDistributeGns(cross)

  // Attempt to execute the proposal with current context
  proposal, err := executeProposal(
    proposalID,
    currentAt,
    currentHeight,
    caller,
  )
  if err != nil {
    panic(err)
  }

  // Emit execution event for tracking and auditing
  previousRealm := std.PreviousRealm()
  std.Emit(
    "Execute",
    "prevAddr", previousRealm.Address().String(),
    "prevRealm", previousRealm.PkgPath(),
    "proposalId", formatInt(proposalID),
  )

  return proposal.ID()
}
```

Core governance operations, including voting, execution, and reset, are implemented to call `emission.MintAndDistributeGns`, which internally invokes `AssertIsNotHaltedEmission`. As a result, governance functions cannot proceed when emission is halted. This design introduces a dependency between emission and governance: in scenarios where emission must be stopped—such as during emergencies—governance functionality also becomes unavailable.

[Recommendation]
The governance logic should be decoupled from emission so that halting emission does not disable governance. One approach is to make emission calls optional or non-blocking, allowing governance actions to continue even if minting and distribution cannot proceed.

[Update]
v2.0 — Fixed

This issue has been resolved in commit 78e8a511cd351c649f1277657252a81d4f248c32, which decouples governance execution from emission minting by preventing emission halt checks from blocking core governance operations, allowing proposals, voting, and execution to proceed even when emission is halted.

11. Informational - Duplicate Halt Check in MintGns

gns.gno Go

```go
// MintGns mints new GNS tokens according to the emission schedule.
//
// Parameters:
//   - address: recipient address for minted tokens
//
// Returns amount minted.
// Only callable by emission contract.
func MintGns(cur realm, address std.Address) int64 {
  halt.AssertIsNotHaltedEmission()

  ...

  return amountToMint
}
```

The emission halt condition is verified twice in the minting path. emission.MintAndDistributeGns checks the halted status before invoking gns.MintGns, and gns.MintGns repeats the same check. Because MintGns is already guarded by AssertIsEmission, which restricts entry to authorized emission callers, the second halt check does not add meaningful protection. The redundancy does not change behavior but marginally increases gas usage and code complexity.

[Recommendation]

Remove the halt check from gns.MintGns when the function is only entered via MintAndDistributeGns.

[Update]
v2.0 — Fixed
This issue has been resolved in commit 9eb6642d4c7238b10009766f9818bd4d9e67b748, which removes the redundant emission halt check from MintGns to avoid duplicate validation already enforced in MintAndDistributeGns.

Note: If the emission contract is designed to be upgradeable, retaining halt checks at multiple layers (as in the previous design) can be a reasonable defensive measure to guard against unexpected changes in the call path.

12. Informational - Duplicate Condition in SetDistributionStartTime

emission.gno Go

```go
// Must be in the future.
if startTimestamp > 0 && startTimestamp <= currentTimestamp {
    panic("distribution start timestamp must be greater than current timestamp")
}

prevStartTimestamp := distributionStartTimestamp

// Reinitialize emission state if not started yet.
if startTimestamp > currentTimestamp && gns.MintedEmissionAmount() == 0 {
    gns.InitEmissionState(cross, startTimestamp, startTimestamp)
}
```

The `SetDistributionStartTime` function checks that the start timestamp is in the future (`startTimestamp > currentTimestamp`) and later repeats essentially the same condition when deciding whether to reinitialize the emission state. Although the logic is functionally correct, duplicating the condition across separate branches reduces readability and adds unnecessary complexity. The initial check already guarantees that the timestamp is in the future; the second branch only adds the `MintedEmissionAmount() == 0` guard, making the repetition redundant.

[Recommendation]
Simplify the control flow by consolidating the timestamp validation and emission state reinitialization into a single branch. The future timestamp condition should be evaluated once, and the `MintedEmissionAmount() == 0` check can then be nested or chained to determine whether reinitialization is required.

[Update]
v2.0 — Fixed
This issue has been resolved in commit 64c4e56682fd29d5806374796a989ce6d5bf3e97, which simplifies the control flow in SetDistributionStartTime by consolidating duplicate future timestamp checks and clearly separating timestamp validation from emission state reinitialization logic.

13. Informational - Missing Default Address Fallback in transferToTarget

distribution.gno Go

```go
// transferToTarget sends tokens to the appropriate target address.
//
// Returns error if target address not found.
func transferToTarget(target int, amount int64) error {
  switch target {
  case LIQUIDITY_STAKER:
    stakerAddr, ok := access.GetAddress(prbac.ROLE_STAKER.String())
    if !ok {
      return makeErrorWithDetails(
        errDistributionAddressNotFound,
        ufmt.Sprintf("%s not found", prbac.ROLE_STAKER.String()),
      )
    }

    gns.Transfer(cross, stakerAddr, amount)
    distributedToStaker += amount
    accuDistributedToStaker += amount
  }

  ...

  return nil
}
```

The `transferToTarget` function aborts distribution when a target address lookup via `access.GetAddress` fails, returning `errDistributionAddressNotFound` and halting emission. Other modules (e.g., GNS) apply a fallback pattern that substitutes a default role address when PRBAC lookups fail, preserving liveness. The absence of a similar fallback here introduces an avoidable single point of failure. Misconfiguration or incomplete setup of role addresses can stop all mint-and-distribute flows, disrupting token distribution and dependent protocol operations.

[Recommendation]
Adopt a consistent fallback strategy by supplying a default role address when `access.GetAddress` returns false, mirroring the GNS module's behavior. Ensure the fallback path is explicit in state and logs for auditability, and add tests that cover three cases per target: configured address present, address missing with fallback, and invalid target.

[Update]
v2.0 — Fixed
This issue has been resolved in commit 2d97b61d75d6695a96d61897df14585450f2c305, which removes the implicit fallback behavior in transferToTarget and enforces explicit configuration of distribution target addresses.

14. Informational - Quorum Definition as "Absolute Yea Votes"

proposal_vote_status.gno Go

```go
func (p *ProposalVoteStatus) IsPassed() bool {
    return p.yea >= p.quorumAmount
}
```

The governance quorum is implemented as an absolute threshold of affirmative ("yea") votes rather than a participation-based percentage. Under this model, a proposal can pass with relatively low overall turnout as long as the absolute yea threshold is met. For instance, with a quorum of 1,000 yea votes, a proposal would pass even if only 1,200 total votes were cast, despite representing a small fraction of the total eligible voting power.

This approach differs from practices in other governance systems. Uniswap, for example, also adopts an **absolute yea-vote quorum**: proposals require at least 40 million UNI (approximately 4% of total supply) in affirmative votes to pass, regardless of turnout. By contrast, Arbitrum DAO defines quorum as a **percentage of total voting supply**.

[Recommendation]
Clarify in documentation whether the absolute-yea quorum is intentional and consistent with the protocol's governance philosophy. If intentional, provide rationale and examples to set expectations for voters. If not, consider adopting a participation-based quorum system that requires a minimum turnout percentage in addition to majority approval, and update on-chain checks and documentation accordingly.

[Update]
v2.0 — Fixed
This issue has been resolved in commit d94ae2bf2c78daf815b25e0082a85f7c27e7fe5a, which clarifies through documentation and comments that governance quorum is intentionally defined as an absolute threshold of affirmative votes.

15. Informational - Function Name Typo

governance_propose.gno Go

```go
func getUserVotingInfotWithDelegationSnapshots(
    snapshotTime int64,
) (map[string]*VotingInfo, int64, bool) {
```

The delegation snapshot access function is misspelled as `getUserVotingInfotWithDelegationSnapshots` instead of the intended `getUserVotingInfoWithDelegationSnapshots`.

[Recommendation]
Rename the function to `getUserVotingInfoWithDelegationSnapshots` to correct the typo and maintain consistency with naming conventions. Update all references accordingly to ensure clarity and ease of future maintenance.

[Update]
v2.0 — Fixed
This issue has been resolved in commit 06e4f73d0c653efe29af1a3ef405fb2af788a624, which corrects a typo.

16. Low - Permanent Lockup of Remaining Project Rewards Due to Non-Withdrawal by Minor Users

The `TransferLeftFromProjectByAdmin` function, which allows the Admin to reclaim remaining reward tokens (undistributed amount) after the project ends, is designed to always fail if `tier.CurrentDepositCount() > 0`.

While this is a safeguard to prevent the Admin from stealing funds when user deposits remain, conversely, if even one user neglects to withdraw (or loses their key), all remaining reward tokens for the project are permanently locked up.

For example, if 100,000 GNS was set as a reward, but only 1,000 GNS was distributed due to low participation, leaving 99,000 GNS remaining, if one small depositor doesn't withdraw, the 99,000 GNS can never be reclaimed.

[Update]
v2.0 — Fixed
This issue has been resolved in commit e53942f36b5786e17f07301c32c63db483de71de, which prevents permanent lockup of remaining project rewards by allowing the admin to reclaim undistributed tokens after project completion.

17. Informational - Insufficient Validation When Proposing Governance Parameter Changes

The validateParameterChange function permits the creation of proposals that target non-existent functions or invalid package paths without any verification against the ParameterRegistry. Consequently, these zombie proposals may pass the voting process but will inevitably fail during execution, wasting governance attention and resources on functional changes that can never be applied.

[Recommendation]
Implement validation at the proposal creation stage to check if the target function is registered in the ParameterRegistry.

[Update]
v2.0 — Fixed
This issue has been resolved in commit 10f01f0f37d0c3dda59ece6d26f11b5770fccf11, which adds validation during governance proposal creation to ensure that parameter change proposals reference registered package paths and valid handler functions.

18. Informational - Insufficient Validation of Token Path in Community Pool Spend Proposals

The `validateCommunityPoolSpend` function validates the recipient address and amount but fails to verify if the requested tokenPath is a valid, registered token in the GRC20 registry. This enables the spamming of the governance system with requests for invalid or fake tokens, cluttering the proposal list with funding requests that have no legitimate asset backing.

[Recommendation]
Add a check using `common.MustRegistered(tokenPath)` during proposal creation to ensure the requested token exists in the registry.

[Update]
v2.0 — Fixed
This issue has been resolved in commit 57c6936426779e02a7263b167e700144f5cc2c76, which adds validation when creating community pool spend proposals.